Recursive Probabilistic Estimation of Dynamic Graph Edge Weights from Node Signals

Gurui Jin and Ercan E. Kuruoğlu

***Abstract*—Dynamic graph tracking requires estimating both the existence and strength of edges. We study recursive probabilistic estimation of dynamic edge weights from node signals. Node dynamics yield an edge-weight regression; Markov prediction and mean-field updating maintain edge probabilities. A support decision followed by conditional amplitude estimation produces a Laplacian and its complete spectrum. On 24 synthetic condition trajectories comprising 1,728 blocks, mean relative edge-weight and eigenvalue errors decrease from 0.15134 and 0.08741 for a task-adapted hard-threshold sparse EKF to 0.12163 and 0.06527. False splits of connected graphs decrease from 76/1068 to 8/1068. These improvements come with greater computation time and worse event-block increment error. We evaluate this tradeoff through the structural, weight, and spectral outputs of a single estimator.**

***Index Terms*—Dynamic graph learning, edge-weight estimation, recursive inference, mean-field approximation, graph spectra.**

I. Introduction

GRAPH signal processing describes interactions between nodes and uses Laplacian spectra to characterize signal variation [1]. When edges disappear, reappear, and change strength, online inference must distinguish absent edges from weak connections while tracking their amplitudes. We ask how observed node trajectories, under a specified interaction model, can provide recursive edge estimates and consistent spectral outputs U_t, Λ_t.

Natali *et al.* study time-varying optimization for online graph learning [2]. Dabush *et al.* treat edge weights as latent states and graph-filtered signals as observations in a sparsity-aware EKF [3]. Edge-weight Laplacian parameterization and the linear special case of first-order graph filtering are established foundations. Our coordinate update likewise uses standard mean-field variational inference [4].

The estimator constructs an edge-weight likelihood from successive node observations, recursively maintains a discrete distribution for each edge, and uses it to estimate connections and their strengths. It combines Markov prediction, mean-field updating, and a point-estimate readout. Distributions carry information from one block to the next. We evaluate estimation error, structural recovery, and computational cost against literature-derived baselines.

Gurui Jin and Ercan E. Kuruoğlu are with Shenzhen International Graduate School, Tsinghua University, Shenzhen, China (e-mail: jgr23@mails.tsinghua.edu.cn; kuruoglu@sz.tsinghua.edu.cn).

II. From Node Observations to Edge Information

A. Unknowns, observations, and modeling rationale

Let $s_{t,k} \in \mathbb{R}^n$ contain scalar node signals. Index t denotes a graph-update block and k a within-block sample. The unknown $\theta_t \in [0,1]^m$ contains candidate *edge* weights, not node values. Each column of $B = [b_1, \ldots, b_m]$ has $+1, -1$ at the endpoints of one edge and zeros elsewhere, giving

$$L(\theta_t) = B\,\mathrm{diag}(\theta_t)B^{\mathsf{T}}. \tag{1}$$

The arbitrary orientation does not affect $b_e b_e^{\mathsf{T}}$. We use the first-order local interaction model

$$s_{t,k+1} = (aI - \eta L(\theta_t))s_{t,k} + u_{t,k} + \epsilon_{t,k}. \tag{2}$$

At node i, the interaction is $-\eta \sum_j \theta_{ij}(s_{i,k} - s_{j,k})$, added to self-evolution $as_{i,k}$ and known input $u_{i,k}$. This discrete diffusion-type graph filter [1] applies when signals follow such coupling. All states are synchronously observed; a, η, u, σ are known. Process innovations are independent across transitions, independent of the current state, and distributed as $\mathcal{N}(0, \sigma^2 I)$. There is no additional sensor measurement noise.

For intuition, consider room temperatures coupled by heat exchange. Edge weights describe exchange strengths, while $as + u$ predicts the next temperatures without inter-room exchange. The difference from the observed next temperatures contains coupling and disturbance. Repeated temperature changes therefore inform the unknown strengths.

B. Residual meaning and derivation

The regression response is computed from observations:

$$r_{t,k} = as_{t,k} + u_{t,k} - s_{t,k+1}. \tag{3}$$

Here $s_{t,k}$ is the current observed node-signal vector; a is the known self-evolution coefficient; $as_{t,k}$ predicts

self-evolution alone; $u_{t,k}$ is the known input over the sampling interval; and $s_{t,k+1}$ is the next observation. Adding the first two terms predicts the next state without neighbor interactions. Subtracting the observation gives $r_{t,k}$. For a node with current value 1, $a = 0.9$, input 0.02, and next observation 0.82, the residual is $0.9 \times 1 + 0.02 - 0.82 = 0.10$. It contains neighbor coupling and disturbance. Suppressing indices, substitution gives

$$\begin{aligned} r &= as + u - [(aI - \eta L)s + u + \epsilon] \\ &= \eta L s - \epsilon \\ &= \eta B \operatorname{diag}(\theta) B^{\mathsf{T}} s - \epsilon \\ &= \underbrace{\eta B \operatorname{diag}(B^{\mathsf{T}} s)}_{H} \theta - \epsilon. \end{aligned} \tag{4}$$

The last step uses $\operatorname{diag}(\theta)d = \operatorname{diag}(d)\theta$ for $d = B^{\mathsf{T}}s$. Column e is $\eta b_e(b_e^{\mathsf{T}}s)$: an endpoint indicator scaled by the endpoint signal difference and coupling coefficient. Once s is observed, H is known and the equation is linear in θ.

Before observation, the innovation and unknown weights make the next state and residual random. After observation, the residual is numerically determined. These statements are compatible: the likelihood describes how the observed value depends on unknown weights. Edge estimation combines the interaction model with information from successive state transitions.

C. Three-node example and block stacking

For candidate edges $(1,2),(1,3),(2,3)$, take $s = (1, 0, 0.5)^{\mathsf{T}}$, $a = 0.9$, $\eta = 0.1$, and $u = 0$. Then

$$B = \begin{bmatrix} 1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & -1 & -1 \end{bmatrix}, \quad H = \begin{bmatrix} .1 & .05 & 0 \\ -.1 & 0 & -.05 \\ 0 & -.05 & .05 \end{bmatrix}. \tag{5}$$

For example, the second column is $0.1(1, 0, -1)^{\mathsf{T}}(1 - 0.5)$. Generating a noiseless example with $\theta = (0.8, 0.5, 0)^{\mathsf{T}}$ gives $H\theta = (.105, -.08, -.025)^{\mathsf{T}}$ and $s_{k+1} = (.795, .08, .475)^{\mathsf{T}}$. An observer computes the same residual as $0.9s - s_{k+1}$ without knowing the generating weights. Further within-block observations supply more equations for the same edge weights.

Keeping θ_t fixed for K transitions provides K equations. Stacking places them vertically, without averaging:

$$\underbrace{\begin{bmatrix} r_{t,0} \\ \vdots \\ r_{t,K-1} \end{bmatrix}}_{r_t \in \mathbb{R}^{nK}} = \underbrace{\begin{bmatrix} H_{t,0} \\ \vdots \\ H_{t,K-1} \end{bmatrix}}_{H_t \in \mathbb{R}^{nK \times m}} \theta_t - \epsilon_t. \tag{6}$$

Each transition adds constraints on the same weights. Equal endpoint signals give $B^{\mathsf{T}}s = 0$ and $H = 0$, conveying no instantaneous edge information. Full column rank of the stacked design suffices for uniqueness of unconstrained noiseless regression; a prior does not replace missing observation information.

III. Recursive Edge-Distribution Estimation

A. Finite representation and prediction

Each edge takes values $v_j = j/64$, $j = 0, \ldots, 64$, with approximate probabilities $q_{t,e}(j)$. Here $v_0 = 0$ means no edge, while positive values describe connection strengths. Prediction is

$$p^{-}_{t,e}(j) = \sum_{i=0}^{64} q_{t-1,e}(i) T_{ij}, \tag{7}$$

where T_{ij} describes a transition between weight states. The experiments fix zero-to-positive and positive-to-zero switching probabilities to 0.01 per block, using a discretized Gaussian transition for positive amplitudes. The first block uses the initial distribution with zero-state probability 0.5; subsequent blocks each perform one prediction.

B. Why mean-field, and how it updates

Exact enumeration requires 65^m joint configurations. The factorization $q_t(\theta_t) = \prod_e q_{t,e}(\theta_{t,e})$ stores $65m$ probabilities. Interactions enter each update through current means, trading a general representation of inter-edge dependence for tractable computation [4].

Multiplying the conditional transition densities along the observed trajectory yields the log likelihood

$$\begin{aligned} \ell_t(\theta_t) &= -\tfrac{1}{2}\theta_t^{\mathsf{T}} G_t \theta_t + h_t^{\mathsf{T}} \theta_t + C, \\ G_t &= H_t^{\mathsf{T}} H_t / \sigma^2, \quad h_t = H_t^{\mathsf{T}} r_t / \sigma^2. \end{aligned} \tag{8}$$

This sequential likelihood does not assume that the entire trajectory-dependent design is independent of all process innovations. Let $\mu_{t,f} = \sum_j v_j q_{t,f}(j)$. The standard rule $\log q_{t,e} = \log p^{-}_{t,e} + \mathbb{E}_{-e}\ell_t + \text{const}$ gives

$$\begin{aligned} q_{t,e}(j) &\propto p^{-}_{t,e}(j) \exp[-\tfrac{1}{2} G_{t,ee} v_j^2 + c_{t,e} v_j], \\ c_{t,e} &= h_{t,e} - \sum_{f \neq e} G_{t,ef} \mu_{t,f}. \end{aligned} \tag{9}$$

The cross term $-G_{ef}\theta_e\theta_f$ becomes $-G_{ef}v_j\mu_f$ after taking the expectation; terms independent of v_j enter the normalizer. Updates use a fixed edge order, normalize each distribution, and stop when maximum mean and probability changes both fall below 10^{-8}, or after 30 sweeps. Capped blocks are retained and recorded.

C. One estimator and one point-output rule

We fix the support readout

$$\hat{\theta}_{t,e} = \begin{cases} 0, & q_{t,e}(0) \geq 1/2, \\ \dfrac{\sum_{j=1}^{64} v_j q_{t,e}(j)}{1 - q_{t,e}(0)}, & q_{t,e}(0) < 1/2. \end{cases} \tag{10}$$

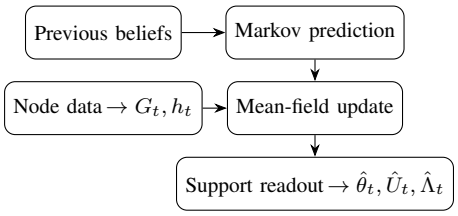


Fig. 1. Prediction, observation update, and output of the estimator. Updated beliefs carry to the next block.

Algorithm 1 Recursive edge estimation from node signals

Require: Node data, known u, a, η, σ, B, T, and initial beliefs
for each block t **do**
  Use initial beliefs at the first block; otherwise predict by (7)
  Form r_t, H_t by (3), (4), then G_t, h_t
  Initialize q from predicted beliefs and compute edge means
  for at most 30 sweeps over edges in fixed order **do**
    Update and normalize each $q_{t,e}$ by (9); refresh its mean
    Stop after a sweep if probability and mean changes are both $\leq 10^{-8}$
  end for
  Read out $\hat{\theta}_t$ by (10); retain q_t for the next block
  Form $\hat{L}_t$; output $\hat{U}_t, \hat{\Lambda}_t$ by full EVD
end for

This makes an existence decision followed by conditional amplitude estimation. For example, $q(0) = 0.7$ produces zero instead of a small positive mixture mean. The output first decides existence, then estimates strength conditional on existence.

Equation (1) then gives $\hat{L}_t$, and a full eigendecomposition yields $\hat{L}_t = \hat{U}_t \hat{\Lambda}_t \hat{U}_t^{\mathsf{T}}$. The recursive state is the edge distribution. Full EVD of the point graph produces U, Λ as derived outputs for spectral-error evaluation; they do not enter the edge-distribution update.

D. Relationship to existing filters

Our estimator and KF share Bayesian prediction and updating. Gaussian KF represents a continuous state by its mean and covariance; here, an explicit zero mass and positive grid probabilities are updated using (9). Gaussian process innovations do not imply a single Gaussian edge posterior. The distinction is not observation nonlinearity: (4) is exactly linear in edge weights and needs no EKF Taylor approximation. Dabush *et al.* also address more general nonlinear graph filters [3].

IV. Experiments and Existing-Method Comparisons

A. Common task and metrics

We use 6/12 nodes, two structural-change generators, two excitation conditions, and three graph draws per condition: 24 trajectories of 72 blocks, each with $K = 48$ transitions. All $m = n(n-1)/2$ candidate edges are available. Parameters are $a = 0.95$, $\eta = 0.8/n$, and $\sigma = 0.03$, with zero-mean Gaussian input standard deviation 0 or 0.3. Graph and observation seeds were fixed before testing; estimator parameters carry over from development. Paired excitation conditions share graphs, so the 1,728 blocks are not independent replicates.

TABLE I
Same-observation graph estimation. Lower errors and false splits are better; more correct partitions are better. Column-wise best values are bold.

Method	E_θ	E_λ	Partitions	False splits
Proposed	**.12163**	**.06527**	404/660	8/1068
Sparse EKF adapted	.15134	.08741	330/660	76/1068
ProbSSM average	.16894	.08844	2/660	**0/1068**
ProbSSM local MAP	.19677	.10135	**545/660**	103/1068

Literature-derived baselines. The main comparison adapts the hard-threshold branch of the public implementation accompanying Dabush *et al.* [3], [5] to the same regression, with $[0, 1]$ constraints and the previously selected threshold 0.1. We call it the sparse EKF adaptation, distinguishing it from the paper's principal ℓ_1 algorithm. The supplementary ProbSSM baseline comes from the same authors' repository [5]. It maintains node-local binary incident-edge configurations with fixed positive amplitude 1. Its average and local-MAP readouts share inference. It is a repository baseline, not another proposed method or a substitute for the paper's main algorithm.

Every method accesses the same observed signals and known parameters, and computes a full spectrum from its point graph. For ascending eigenvalues λ_t, blockwise errors are

$$E_\theta(t) = \frac{\|\hat{\theta}_t - \theta_t\|_2}{\max(\|\theta_t\|_2, 10^{-15})}, \qquad E_\lambda(t) = \frac{\|\hat{\lambda}_t - \lambda_t\|_2}{\max(\|\lambda_t\|_2, 10^{-15})}. \tag{11}$$

We report means across blocks. Correct partition recovery requires all connected components to match on a disconnected true graph. A false split occurs when a connected true graph is estimated as disconnected. Their denominators are 660 and 1068 blocks, respectively. Components use strictly positive edges in the point graph.

B. Accuracy, change error, and cost

Relative to the sparse EKF adaptation, mean edge and eigenvalue errors decrease by about 19.6% and 25.3%, with improved aggregate partition counts. ProbSSM local MAP recovers more disconnected partitions but produces more false splits. ProbSSM was added on previously used observations; its fixed amplitude

mismatches multivalued true weights, and its parameters were not independently optimized.

For blocks with true support changes, we average squared point-increment errors over all edges and event blocks, then take the square root. The proposed method scores 0.11435 versus 0.09379 for the sparse EKF adaptation: improved average estimation does not also improve abrupt-change response. Single-thread online time, including statistics, inference, and full EVD, averages 9.75 versus 1.04 seconds per trajectory. Initialization, warmup, storage, and evaluation are excluded; the machine was shared. All 14/1728 capped proposed-method blocks remain in the results.

Under the archived common-gap selection, projection errors over 5,517 positive simple eigenvalue directions average 0.86552 (proposed) and 0.87621 (sparse EKF adaptation). Eligibility depends on truth and all outputs of the original experiment, excludes repeated eigenspaces, and does not establish a general advantage for the complete U.

C. Synthetic UAV altitude coordination

Consider high-level altitude coordination of six UAVs, with $s_i = z_i - z_{\mathrm{ref}}$ denoting deviation from a 30-m reference altitude. A first-order altitude response with neighbor consensus feedback [6], discretized by Euler with $\Delta = 0.1$ s, gives

$$\begin{aligned} s_{i,k+1} = (1 - \kappa\Delta)s_{i,k} + u_{i,k} + \epsilon_{i,k} \\ - c\Delta \sum_j \theta_{ij}(s_{i,k} - s_{j,k}). \end{aligned} \tag{12}$$

Set $\kappa = 0.5$ and $c\Delta = 0.8/6$, recovering (2). The input u_i is a known altitude-control increment and edge weights are effective neighbor-feedback strengths. Prescribed horizontal trajectories are held fixed within each block. Horizontal distance in meters sets $\theta_{ij} = \mathbf{1}[d_{ij} < 35] \exp[-(d_{ij}/25)^2]$. Estimators receive only altitude signals, inputs, and known parameters.

Six new geometry seeds, 720110–720115, each use input standard deviations 0 and 0.3: 12 trajectories of 36 blocks, 432 blocks in total, with 48 transitions per block. Estimator parameters remain unchanged. The two input conditions share geometry; all 3/432 capped proposed-method blocks are retained. All nodes are observed synchronously under an ideal first-order altitude response. Table II reports this synthetic-model experiment.

To evaluate decision use, each block's same terminal state s is supplied to controllers using the proposed estimate, the sparse EKF estimate, no graph, and the true graph. Minimizing predicted altitude error plus control effort gives $v = \mathrm{clip}(-\hat{F}s/2, -1, 1)$, with $\hat{F} = aI - \eta\hat{L}$; the no-graph model uses aI. A shared true model and disturbance w score $J = \|Fs + v + w\|^2 + \|v\|^2$. With input standard deviation 0.3, mean costs are 0.701610, 0.701721, 0.745394, and 0.701512, respectively. The proposed estimate reduces cost by 5.87% relative to no graph, but only 0.016% relative to sparse EKF. With zero input, costs are 0.011256, 0.011258, 0.011698, and 0.011236. This is matched-state one-step decision replay: actions do not feed subsequent observations. The spectral expression $\hat{F} = \hat{U}(aI - \eta\hat{\Lambda})\hat{U}^{\mathsf{T}}$ is equivalent to direct use of $\hat{L}$; the experiment evaluates use of the estimated graph.

TABLE II
SYNTHETIC HIGH-LEVEL UAV MODEL: RELATIVE ERRORS AND TIME.

Input std.	Method	E_θ	E_λ	ms/block
0	Proposed	**.33687**	**.11018**	2.92
0	Sparse EKF	.39423	.15612	**.56**
0.3	Proposed	**.07065**	**.02180**	3.30
0.3	Sparse EKF	.09069	.03621	**.59**

V. CONCLUSION

The estimator constructs an edge-weight regression from successive node observations and recursively maintains edge distributions through Markov prediction and mean-field updating. It outputs dynamic connections and their strengths. In the original synthetic comparison, mean weight and eigenvalue errors decrease by 19.6% and 25.3% relative to the sparse EKF adaptation, while false splits decrease from 76/1068 to 8/1068, with greater computational cost and event-increment error. The added altitude-coordination simulation also yields lower average estimation errors; its one-step control cost is close to sparse EKF. These results characterize the estimator's accuracy, cost, and dynamic-response tradeoff.

REFERENCES

[1] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst, "The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains," *IEEE Signal Process. Mag.*, vol. 30, no. 3, pp. 83–98, 2013.

[2] A. Natali, E. Isufi, M. Coutino, and G. Leus, "Learning time-varying graphs from online data," *IEEE Open J. Signal Process.*, 2022, doi: 10.1109/OJSP.2022.3178901.

[3] L. Dabush, N. Shlezinger, and T. Routtenberg, "Sparsity-aware extended Kalman filter for tracking dynamic graphs," arXiv:2507.09999, 2025.

[4] D. M. Blei, A. Kucukelbir, and J. D. McAuliffe, "Variational inference: A review for statisticians," *J. Amer. Statist. Assoc.*, vol. 112, no. 518, pp. 859–877, 2017.

[5] L. Dabush, "PolyEKF," source snapshot cbb1a0a (full commit recorded with the experiments), `baseline_prob_ssm.py` and EKF implementation. [Online]. Available: https://github.com/lital-dgold/PolyEKF

[6] R. Olfati-Saber and R. M. Murray, "Consensus problems in networks of agents with switching topology and time-delays," *IEEE Trans. Autom. Control*, vol. 49, no. 9, pp. 1520–1533, 2004, doi: 10.1109/TAC.2004.834113.